

Proyecto Integrador: CatálogoBulk

Sistema de importación masiva de productos con procesamiento asíncrono

Especificación completa. Todo lo aquí definido es contrato: nombres de endpoints, campos, estados, eventos y códigos de error deben implementarse **tal cual**. Donde algo es decisión del aprendiz, se marca explícitamente como *(libre)*.

1. Escenario y objetivo

1.1 Contexto de negocio

Una tienda de e-commerce vende productos de **múltiples proveedores**. Cada proveedor le envía periódicamente su catálogo completo en un archivo (CSV o JSON) con miles de referencias. Hoy alguien los mete a mano: inviable y lleno de errores.

El objetivo del sistema es **automatizar la ingesta de esos catálogos**: que un admin suba el archivo del proveedor y el sistema incorpore esos productos al **catálogo consultable de la tienda** de forma rápida, confiable y trazable, sin bloquear la operación mientras procesa.

1.2 Objetivo del procesamiento (qué produce el worker)

El procesamiento en segundo plano es el corazón del proyecto. No es "insertar filas": es una **tubería que transforma un archivo crudo del proveedor en un catálogo limpio y consultable**, más un reporte de qué pasó.

Entrada	Un archivo del proveedor (CSV/JSON) con filas potencialmente sucias: campos vacíos, precios mal formateados, duplicados, mayúsculas inconsistentes.
Transformación	Por cada fila: validar contra las reglas de negocio (sección 6.3), normalizar los datos válidos (sección 6.4), y persistir en el catálogo. Las inválidas se descartan y se registran con su motivo.
Salida	1) El catálogo de productos actualizado y listo para consultar vía GET /api/productos y /stats. 2) Un reporte del import (ImportJob): total, exitosos, fallidos, y el detalle de cada error.
Valor	El admin pasa de "meter productos a mano" a "subir un archivo y en segundos tener el catálogo actualizado, sabiendo exactamente cuántos entraron, cuántos fallaron y por qué".

El ciclo se cierra así: importar es lo que *puebla* el catálogo. Los endpoints de consulta (GET

/api/productos, /stats) leen justamente lo que el procesamiento dejó. Por eso la caché (Fase 5) tiene que invalidarse cuando termina un import: los datos que el usuario consulta acaban de cambiar.

1.3 Restricción técnica que da forma al diseño

Procesar el archivo dentro de la petición HTTP bloquearía el servidor y daría timeout con archivos grandes. De ahí salen los cinco pilares del sistema:

1. Recibir el archivo y responder de inmediato (sin procesarlo).
2. Procesarlo en segundo plano, por lotes.
3. Reportar progreso en tiempo real.
4. Cachear las consultas pesadas e invalidarlas cuando los datos cambian.
5. Correr todo en contenedores.

2. Stack fijo

- **Runtime:** Node.js 20 LTS
- **Framework HTTP:** Express
- **Base de datos:** MongoDB 7 (driver: Mongoose)
- **Cola / caché / pub-sub:** Redis 7
- **Cola de jobs:** BullMQ
- **Tiempo real:** Socket.io
- **Uploads:** Multer
- **Auth:** JWT + bcrypt
- **Tests:** Jest + Supertest
- **Docs:** Swagger (swagger-ui-express + OpenAPI 3)
- **Contenedores:** Docker + Docker Compose

Versiones de librerías del ecosistema Node: (*libre*), pero deben quedar fijadas en `package.json`.

3. Variables de entorno (.env)

Todas obligatorias. El proyecto **no debe** arrancar con valores hardcodeados.

```
PORT=3000
MONGO_URI=mongodb://mongo:27017/catalogobulk
REDIS_HOST=redis
REDIS_PORT=6379
JWT_SECRET=cambiar_en_produccion
JWT_EXPIRES_IN=1h
MAX_FILE_SIZE_MB=50
BATCH_SIZE=500
```

```
CACHE_TTL_SECONDS=300
IMPORT_ERRORS_CAP=1000
```

- BATCH_SIZE: filas por lote en el bulk insert.
- MAX_FILE_SIZE_MB: límite de tamaño de archivo aceptado.
- IMPORT_ERRORS_CAP: máximo de errores individuales que se guardan por job (para no reventar el documento en Mongo).

4. Estructura de carpetas (obligatoria)

Arquitectura en capas por módulo. controller → service → repository. El controller nunca habla con Mongoose directo; el repository nunca sabe de req/res.

```
src/
  config/
    db.js           # conexión Mongo
    redis.js        # cliente Redis compartido
    env.js          # lee y valida .env (falla al arrancar si falta algo)
  middlewares/
    auth.js         # verifica JWT, adjunta req.usuario
    rol.js          # exige rol específico
    errorHandler.js # manejador de errores centralizado (último middleware)
    upload.js       # config de Multer (diskStorage)
    rateLimit.js    # limitadores
  modules/
    auth/
      auth.controller.js
      auth.service.js
      auth.routes.js
    productos/
      producto.model.js
      producto.controller.js
      producto.service.js
      producto.repository.js
      producto.routes.js
    proveedores/
      proveedor.model.js
      proveedor.controller.js
      proveedor.service.js
      proveedor.repository.js
      proveedor.routes.js
    categorias/
      categoria.model.js
      categoria.controller.js
      categoria.service.js
      categoria.repository.js
      categoria.routes.js
    imports/
      importJob.model.js
      import.controller.js
      import.service.js
      import.repository.js
      import.routes.js
  scripts/
    generar-catalogo.js # genera el archivo de prueba (≥120.000 filas sucias)
  queues/
    import.queue.js    # define la Queue de BullMQ
  workers/
```

```

import.worker.js # proceso separado; procesa los jobs
sockets/
  index.js       # inicializa Socket.io y el relay QueueEvents→socket
errors/
  AppError.js    # clase de error tipada (statusCode, mensaje, código)
app.js          # construye la app Express (sin listen)
server.js       # arranca HTTP + Socket.io + conexiones

```

workers/import.worker.js se ejecuta como **proceso aparte** (node src/workers/import.worker.js), no dentro del API server. Esto es central y se explica en la Fase 4.

5. Modelos de datos (exactos)

5.1 Usuario (usuarios)

Campo	Tipo	Reglas
_id	ObjectId	auto
email	String	requerido, único, lowercase, formato email válido
password	String	requerido, guardado como hash bcrypt (nunca en texto plano)
rol	String	enum ["admin", "user"], default "user"
createdAt / updatedAt	Date	timestamps automáticos

password **nunca** se devuelve en ninguna respuesta (usar select: false o excluir en el toJSON).

5.2 Producto (productos)

Campo	Tipo	Reglas
_id	ObjectId	auto
sku	String	requerido, único , con índice
nombre	String	requerido, min 1 char
precio	Number	requerido, >= 0
stock	Number	entero, >= 0, default 0
categoria	String	requerido, min 1 char (slug; ver modelo Categoria)
descripcion	String null	opcional, texto libre
imagenUrl	String null	opcional, URL http(s) válida de la imagen principal
proveedorId	ObjectId	ref proveedores, requerido (se asigna en el import)
disponible	Boolean	derivado: true si stock >

Campo	Tipo	Reglas
		0, default según stock
createdAt / updatedAt	Date	timestamps

Índice único en sku. Índices secundarios en categoria y proveedorId (se filtra por ambos).

imagenUrl y descripcion existen para que un cliente del API pueda mostrar el producto con imagen. Son opcionales: un catálogo puede venir sin ellas. disponible lo calcula el backend a partir del stock, para que el cliente no tenga que hacer esa cuenta.

5.3 Proveedor (proveedores)

Representa la marca/distribuidor que envía un catálogo. Cada importación pertenece a un proveedor y cada producto queda asociado a él.

Campo	Tipo	Reglas
_id	ObjectId	auto
nombre	String	requerido, único
slug	String	requerido, único, minúsculas sin espacios (ej: acme-corp)
contactoEmail	String null	opcional, formato email
logoUrl	String null	opcional, URL http(s) del logo
activo	Boolean	default true
createdAt / updatedAt	Date	timestamps

Un proveedor activo: false no puede recibir nuevas importaciones (se rechaza en el endpoint de import).

5.4 Categoria (categorias)

Metadatos de cada categoría del catálogo. El producto guarda la categoría como slug string (barato, no rompe el import por streams); esta colección guarda la información para mostrarla bien. Se **auto-genera** durante el import: por cada categoría nueva que aparezca en un archivo, se hace `upsert` con valores por defecto.

Campo	Tipo	Reglas
_id	ObjectId	auto
slug	String	requerido, único, minúsculas (ej: electronica) — coincide con <code>Producto.categoria</code>
nombre	String	requerido; al auto-crearse desde el import = el slug capitalizado
descripcion	String null	opcional, editable por admin

Campo	Tipo	Reglas
imagenUrl	String null	opcional, URL http(s) de la categoría
createdAt / updatedAt	Date	timestamps

El import solo **crea** categorías que falten con datos mínimos. Enriquecerlas (imagen, descripción bonita) es tarea manual del admin vía PUT /api/categorias/:id.
 Así el catálogo siempre tiene una lista de categorías con metadatos consultable vía API.

5.5 ImportJob (import_jobs)

Representa una importación. Es la fuente de verdad del estado.

Campo	Tipo	Reglas
_id	ObjectId	auto; este id es el importJobId que se expone al cliente
usuarioId	ObjectId	ref usuarios, requerido
proveedorId	ObjectId	ref proveedores, requerido (se elige al subir el archivo)
archivoNombre	String	nombre original del archivo subido
archivoRuta	String	ruta en disco donde Multer lo guardó
estado	String	enum ["pending", "processing", "completed", "failed"], default "pending"
total	Number null	total de filas del archivo; null hasta que se conozca
procesados	Number	filas ya procesadas, default 0
exitosos	Number	filas insertadas OK, default 0
fallidos	Number	filas rechazadas, default 0
errores	Array	de { fila: Number, sku: String\ null, motivo: String }, cap a IMPORT_ERRORS_CAP
bullJobId	String null	id del job en BullMQ (para trazabilidad)
motivoFallo	String null	si estado === "failed", el porqué
startedAt	Date null	cuándo el worker empezó
finishedAt	Date null	cuándo terminó (completed o failed)

Campo	Tipo	Reglas
createdAt / updatedAt	Date	timestamps

Máquina de estados (transiciones válidas, cualquier otra es un bug):

```
pending —(worker toma el job)—> processing —(termina bien)—> completed
                                     └─(error irrecoverable)—> failed
```

- pending: creado y encolado, ningún worker lo tomó.
- processing: worker activo; startedAt seteado.
- completed: terminó de recorrer el archivo; finishedAt seteado. Puede tener fallidos > 0 y aun así estar completed (filas malas ≠ fallo del job).
- failed: error que impide continuar (archivo corrupto/ilegible, header inválido). motivoFallo y finishedAt seteados.

6. Formato del archivo de catálogo (exacto)

6.1 CSV

Primera línea = header **obligatorio y exacto**. Las primeras cinco columnas son obligatorias; descripcion e imageUrl son **opcionales** (pueden venir vacías o el archivo puede no traerlas):

```
sku,nombre,precio,stock,categoria,descripcion,imageUrl
```

Ejemplo:

```
sku,nombre,precio,stock,categoria,descripcion,imageUrl
SKU-001,Camiseta azul,29.99,100,ropa,Algodón 100%,https://cdn.demo.com/img/sku-001.jpg
SKU-002,Taza cerámica,15.50,0,hogar,,
SKU-003,Audífonos BT,89.90,25,electronica,Bluetooth
5.3,https://cdn.demo.com/img/sku-003.jpg
```

6.2 JSON

Array de objetos. descripcion e imageUrl son opcionales:

```
[
  { "sku": "SKU-001", "nombre": "Camiseta azul", "precio": 29.99, "stock": 100,
    "categoria": "ropa", "descripcion": "Algodón 100%", "imageUrl":
    "https://cdn.demo.com/img/sku-001.jpg" },
  { "sku": "SKU-002", "nombre": "Taza cerámica", "precio": 15.50, "stock": 0,
    "categoria": "hogar" }
]
```

El tipo se detecta por la extensión del archivo (.csv o .json). Cualquier otra extensión se rechaza en el upload.

El proveedorId **no viene en el archivo**: se elige al subirlo (POST /api/imports) y se aplica a todas las filas de ese import. Un archivo = un

proveedor.

6.3 Reglas de validación por fila

Una fila es **válida** solo si cumple todo:

Campo	Regla
sku	string no vacío
nombre	string no vacío
precio	número, ≥ 0
stock	entero, ≥ 0
categoria	string no vacío
descripcion	opcional; si viene, string (sin límite estricto)
imageUrl	opcional; si viene, debe ser URL <code>http(s)</code> válida — si es inválida, la fila no se rechaza: se guarda el producto con <code>imageUrl: null</code> y se anota en errores como advertencia (motivo "imageUrl inválida, ignorada")

- Fila inválida (en un campo obligatorio) → se cuenta en fallidos, se registra en errores (respetando el cap) con { fila, sku, motivo }, y **se continúa** con las demás.
- sku duplicado dentro del archivo o ya existente en la base → se cuenta como fallido con motivo "sku duplicado". (No se hace upsert en la versión base; ver extensiones.)
- Un imageUrl malo es una advertencia, no un rechazo: el producto es válido, solo se queda sin imagen. Un error descarta la fila; una advertencia no.

6.4 Reglas de normalización (solo filas válidas)

Antes de persistir, cada fila válida se **limpia** para que el catálogo quede consistente sin importar cómo lo mandó el proveedor. Esto es parte del objetivo del procesamiento, no un extra:

Campo	Normalización
sku	trim, pasar a mayúsculas (SKU-001)
nombre	trim, colapsar espacios múltiples internos a uno
precio	redondear a 2 decimales
stock	forzar a entero (<code>Math.trunc</code>)
categoria	trim, pasar a minúsculas (ropa, no Ropa) — para que el filtro por categoría y las stats agrupen bien
descripcion	trim; si queda vacía → null

Campo	Normalización
imagenUrl	trim; si vacía o inválida → null
disponible	derivado: stock > 0

Ejemplo: la fila `sku-001`, `Camiseta Azul`, `29.999`, `100`, `Ropa`, , se persiste como `{ sku: "SKU-001", nombre: "Camiseta Azul", precio: 30.00, stock: 100, categoria: "ropa", descripcion: null, imagenUrl: null, disponible: true }`.

Sin normalizar, "Ropa" y "ropa" serían dos categorías distintas en las stats, y buscar por sku fallaría por diferencias de mayúsculas/espacios. La normalización es lo que hace que el catálogo sea *usable*, no solo *poblado*.

6.5 Efectos secundarios del import (proveedor y categorías)

Además de insertar productos, cada import tiene dos efectos sobre las otras colecciones:

- **Proveedor:** todas las filas del import se asocian al `proveedorId` elegido al subir el archivo. El proveedor ya debe existir y estar activo.
- **Categorías:** por cada slug de `categoria` que aparezca y **no exista** en la colección `categorias`, se hace `upsert` creando la categoría con `nombre = slug capitalizado` y el resto de campos en `null`. Esto puede hacerse recolectando los slugs distintos y haciendo los `upserts` al final del import (más eficiente que uno por fila). Así el catálogo siempre tiene la metadata mínima de sus categorías, y el admin puede enriquecerla después.

7. Especificación de endpoints (contrato HTTP)

Prefijo base: `/api`. Todas las respuestas son JSON. Errores siguen el formato de la sección 9.

7.1 Auth

POST /api/auth/register — público Request:

```
{ "email": "admin@demo.com", "password": "secret123", "rol": "admin" }
```

- `rol` es opcional; si se omite → `"user"`. Respuestas:
- 201 → `{ "id": "...", "email": "admin@demo.com", "rol": "admin" }`
- 409 → email ya registrado

POST /api/auth/login — público (rate limit estricto) Request: `{ "email": "...", "password": "..." }` Respuestas:

- 200 → `{ "token": "<jwt>" }`
- 401 → credenciales inválidas

El JWT incluye en el payload: `{ sub: usuarioId, rol }`.

7.2 Productos

GET /api/productos — autenticado (cualquier rol) · **cacheable** Query params: page (default 1), limit (default 20, máx 100), categoria (opcional), proveedor (opcional, slug o id), disponible (opcional, true/false). Respuesta 200:

```
{
  "data": [ { "_id": "...", "sku": "SKU-001", "nombre": "...", "precio": 29.99,
"stock": 100, "categoria": "ropa", "descripcion": "Algodón 100%", "imagenUrl":
"https://cdn.demo.com/img/sku-001.jpg", "proveedorId": "660a...", "disponible":
true } ],
  "page": 1,
  "limit": 20,
  "total": 5230
}
```

GET /api/productos/stats — autenticado · **cacheable** Respuesta 200:

```
{
  "totalProductos": 5230,
  "precioPromedio": 42.17,
  "porCategoria": [
    { "categoria": "ropa", "count": 1200 },
    { "categoria": "hogar", "count": 890 }
  ]
}
```

GET /api/productos/:id — autenticado

- 200 → producto (incluye descripcion, imagenUrl, proveedorId, disponible) · 404 → no existe

POST /api/productos — solo admin Request: { "sku", "nombre", "precio", "stock", "categoria", "proveedorId", "descripcion?", "imagenUrl"? }

- 201 → producto creado · 409 → sku duplicado · 400 → validación · 404 → proveedorId no existe

PUT /api/productos/:id — solo admin

- 200 → actualizado · 404 → no existe · 409 → sku duplicado

DELETE /api/productos/:id — solo admin

- 204 → eliminado (sin body) · 404 → no existe

El listado (GET /api/productos) acepta además proveedor (slug o id) y disponible (true/false) como filtros, aparte de categoria. Todos combinables.

7.3 Proveedores

GET /api/proveedores — autenticado (cualquier rol) Query: page, limit, activo (opcional). Respuesta 200: lista paginada.

GET /api/proveedores/:id — autenticado

- 200 → proveedor · 404 → no existe

POST /api/proveedores — solo admin Request: { "nombre", "slug", "contactoEmail"?, "logoUrl"? }

- 201 → creado · 409 → nombre o slug duplicado · 400 → validación

PUT /api/proveedores/:id — solo admin

- 200 → actualizado (permite activo: false para desactivarlo) · 404 → no existe

DELETE /api/proveedores/:id — solo admin

- 409 → **no se puede** eliminar si tiene productos asociados (integridad); usar activo: false en su lugar
- 204 → eliminado si no tenía productos · 404 → no existe

7.4 Categorías

GET /api/categorias — autenticado (cualquier rol) Respuesta 200: lista de categorías con su metadata (slug, nombre, descripcion, imageUrl). Sin paginar (son pocas).

GET /api/categorias/:slug — autenticado

- 200 → categoría · 404 → no existe

PUT /api/categorias/:id — solo admin Request: { "nombre"?, "descripcion"?, "imageUrl"? } (el slug no se edita: es la llave que une con los productos)

- 200 → actualizado · 404 → no existe

No hay POST de categorías: **se crean solas** durante el import (upsert, sección 6.5). El admin solo las **enriquece** con PUT. Puede existir un DELETE opcional, pero borrar una categoría **no** borra los productos que la referencian (quedarían apuntando a un slug sin metadata).

7.5 Imports

POST /api/imports — solo admin · multipart · rate limit estricto

- Campos multipart: archivo (el CSV/JSON) y proveedorId (a quién pertenece el catálogo).
- Multer con diskStorage; límite = MAX_FILE_SIZE_MB.
- Acción: valida extensión y que el proveedorId exista y esté activo, guarda el archivo, crea el ImportJob en pending, lo encola en BullMQ, responde **de inmediato**.

Respuesta 202:

```
{ "importJobId": "665f...", "estado": "pending" }
```

Errores: 400 (sin archivo / extensión inválida / falta proveedorId), 404 (proveedor no existe), 409 (proveedor inactivo), 413 (archivo muy grande).

Este endpoint **no procesa** el archivo. Debe responder en milisegundos.

GET /api/imports/:id — autenticado (dueño o admin) Respuesta 200:

```
{
  "importJobId": "665f...",
  "proveedorId": "660a...",
  "estado": "processing",
  "total": 5000,
  "procesados": 2500,
  "exitosos": 2480,
  "fallidos": 20,
  "porcentaje": 50,
  "errores": [ { "fila": 12, "sku": "SKU-012", "motivo": "precio inválido" } ],
  "startedAt": "2026-07-21T15:00:00.000Z",
  "finishedAt": null
}
```

- `porcentaje = round(procesados / total * 100); 0` si total aún es null.

GET /api/imports — solo admin Lista paginada de import jobs (mismos page/limit que productos).

8. Fases

Cada fase se construye sobre la anterior. No se salta ninguna.

FASE 0 — Setup + Docker Compose

Objetivo: entorno reproducible con un comando.

Entregables:

- Estructura de carpetas de la sección 4.
- `config/env.js` valida que **todas** las variables de la sección 3 existan; si falta una, el proceso hace `process.exit(1)` con mensaje claro (fallar temprano, no a mitad de ejecución).
- `docker-compose.yml` con servicios `api`, `mongo`, `redis`.
- `GET /health` → 200 { `"status": "ok", "mongo": "up", "redis": "up"` }; si alguno falla → 503 con el que está caído en `"down"`.

docker-compose.yml de referencia:

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    env_file: .env
    depends_on:      # solo garantiza ORDEN de arranque, no que estén listos
      - mongo
      - redis

  mongo:
    image: mongo:7
```

```
volumes:
  - mongo_data:/data/db    # persiste datos entre reinicios

redis:
  image: redis:7-alpine

volumes:
  mongo_data:
```

Criterios de aceptación:

- ☐ `docker compose up` levanta los tres servicios.
- ☐ Arrancar sin una variable de entorno → el proceso muere con mensaje explícito.
- ☐ `GET /health` devuelve 503 si Mongo o Redis están caídos, 200 si ambos responden.

Nota: `depends_on` no espera a que Mongo acepte conexiones. El cliente de Mongo debe reintentar la conexión al arrancar, o el primer `/health` fallará.

FASE 1 — Auth + CRUD (productos, proveedores, categorías)

Objetivo: base sólida con lo que ya dominan.

Entregables:

- Modelos Usuario, Producto, Proveedor, Categoría e ImportJob (sección 5).
- Endpoints de auth (7.1) y productos (7.2), **sin** los cacheables todavía (la caché entra en Fase 5; por ahora consultan Mongo directo).
- **CRUD de proveedores (sección 7.3)** — se necesita ya en la Fase 2, porque el import exige un `proveedorId` existente y activo. Sin al menos un proveedor creado, no se puede probar la carga de catálogos.
- **Endpoints de categorías (sección 7.4)** — GET y PUT. La colección estará **vacía** hasta que corra el primer import (Fase 3), que es quien crea las categorías; por ahora los endpoints existen pero devuelven poco. Es correcto así.
- Middleware auth (verifica JWT, adjunta `req.usuario = { id, rol }`) y `rol("admin")`.
- Swagger documentando cada endpoint, sus roles y sus errores.
- Tests de integración: registro, login, acceso denegado a `user` en rutas de admin (productos y proveedores), sku duplicado, integridad al eliminar un proveedor con productos.

Criterios de aceptación:

- ☐ `user` recibe 403 en POST/PUT/DELETE de productos y de proveedores.
- ☐ sku duplicado → 409 tipado, no un 500 crudo de Mongo.
- ☐ slug de proveedor duplicado → 409.

- ☐ password no aparece en ninguna respuesta.
 - ☐ Se puede crear un proveedor y queda `activo: true` por defecto.
 - ☐ Eliminar un proveedor con productos asociados → 409 (integridad).
 - ☐ Swagger levanta en `/api/docs` y describe auth + roles.
-